Federated Learning in SDVs: Optimizing Communication Frequency

Spyridon Arvanitis
Department of Informatics and Telematics
Harokopio University of Athens
Athens, Greece

Abstract*—Training machine learning models across a fleet of Software-Defined Vehicles (SDVs) via Federated Learning (FL) requires balancing limited wireless uplink capacity, client parameter divergence under non-IID data, and processor contention with safety-critical driving tasks. Most existing vehicular FL frameworks bypass this tension by fixing the number of local training epochs and assuming dedicated onboard compute, which frequently saturates the cellular uplink and starves real-time vehicular services of CPU cycles. We present a lightweight runtime policy engine that dynamically determines how many local epochs each vehicle executes and when it uploads parameter updates based on available compute slack, instantaneous link quality, and a short-horizon forecast of local validation loss reduction. Each vehicle transitions among three operational states (`WAIT`, `CONTINUE`, and `SYNC`) governed by a bounded exponential curve-fitting predictor that halts local epochs before diminishing returns set in. We implement the framework in Flower and PyTorch and evaluate it on Dirichlet non-IID CIFAR-10 partitions ($\alpha_{\text{Dir}} \in \{0.1, 0.3, 0.5\}$) driven by time-varying V2X channel and mixed-criticality workload traces against fixed-epoch baselines ($E \in \{1, 5, 10\}$) and FedProx over 250 communication rounds. Under extreme statistical heterogeneity ($\alpha_{\text{Dir}} = 0.1$), Dynamic FedProx achieves 72.02% centralized test accuracy, matching the best static baseline (Fixed FedAvg $E = 5$ at 72.51%) while reducing uplink traffic by 28.2% (saving over 6.5 GB of wireless transfers), cutting transmission buffer overflows by up to 76.0% (from 567 down to 136–140), and lowering compute interference with safety-critical vehicle tasks by up to 73.8%.

***Index Terms*—Federated Learning, Software-Defined Vehicles, V2X Networks, Resource Allocation, Mixed-Criticality Systems.**

I. Introduction

Software-Defined Vehicles (SDVs) consolidate perception, motion planning, and telemetry diagnostics onto centralized onboard computing platforms while continuously generating high-dimensional sensor streams. Federated Learning (FL) provides a natural collaborative training paradigm for SDV fleets, enabling vehicles to jointly refine shared neural networks without transmitting raw, privacy-sensitive driving logs to the cloud.

Deploying FL across an operational vehicular network, however, introduces two system-level bottlenecks that distinguish it from stationary edge environments:

1) **Volatile V2X uplink capacity.** Vehicle-to-Everything (V2X) radio links experience rapid multipath fading, frequent cellular handovers, and shared sector-level uplink congestion. Transmitting multi-megabyte model updates from all participating vehicles at every round rapidly saturates the vehicle-to-infrastructure (V2I) uplink. Conversely, naively reducing upload frequency on non-IID vehicular data accelerates local client drift and degrades the aggregated global model.
2) **Mixed-criticality processor contention.** Unlike dedicated edge servers, SDVs execute FL background tasks on the same Electronic Control Unit (ECU) and System-on-Chip (SoC) hardware that hosts safety-critical Advanced Driver-Assistance Systems (ADAS), such as object detection and trajectory planning. Because FL is strictly best-effort, a rigid, fixed-epoch training schedule that ignores real-time CPU occupancy risks starving high-priority safety pipelines.

Existing vehicular FL schedulers focus primarily on link-layer resource allocation (such as transmit power control, rate adaptation, and client selection) while treating onboard compute as either unconstrained or statically partitioned. Two fundamental limitations persist in the literature. First, conventional schedulers do not evaluate whether scheduled local epochs actually yield meaningful model improvement. Consider an SDV that has completed three local epochs on a skewed non-IID partition and whose validation loss has already plateaued: under a fixed-epoch policy, the vehicle is forced to consume its maximum permitted CPU allocation to execute a fourth epoch anyway, wasting processor cycles, contending with co-located ADAS workloads, and inflating parameter divergence without improving representation quality. Second, prior resource-allocation formulations typically cast FL scheduling as a pure cost-minimization objective:

$$\min \sum_{t=0}^{T-1} \left[\alpha C_{\text{net}}(t) + \beta C_{\text{comp}}(t) + \gamma C_{\text{drift}}(t)\right] \tag{1}$$

where $C_{\text{net}}(t)$, $C_{\text{comp}}(t)$, and $C_{\text{drift}}(t)$ denote communication latency, compute contention, and parameter divergence costs, respectively. Mathematically, the unconstrained global optimum of (1) is the degenerate policy: execute zero local epochs ($e_i(t) = 0$) and transmit zero updates ($p_i(t) = 0$), incurring zero operational cost and achieving zero learning. Existing frameworks avoid this trivial collapse only by imposing hard minimum-epoch or minimum-upload constraints, which reintroduce static scheduling inefficiencies.

To resolve both limitations, we formulate vehicular FL scheduling as an online net-utility maximization problem that

incorporates empirical validation loss reduction $G_i(t)$ directly into the objective:

$$\max \sum_{t=0}^{T-1} \left[\delta \sum_{i \in \mathcal{V}} G_i(t) - \alpha C_{\text{net}}(t) - \beta C_{\text{comp}}(t) - \gamma C_{\text{drift}}(t) \right]$$

Rather than assuming isolated hardware or introducing reinforcement learning training overhead, we design a lightweight deterministic policy engine that monitors real-time safety-workload occupancy, channel throughput, and extrapolated loss decay to transition each vehicle among three operational states: `WAIT` (yield processor resources to safety tasks), `CONTINUE` (train locally while persisting weights on disk across rounds), and `SYNC` (upload parameter updates to the server). In contrast to adaptive asynchronous schemes such as Wu et al. [11] that restrict clients to alternating between local training and uploading, the explicit `WAIT` state prevents vehicles from wasting energy or contending with ADAS workloads when wireless links are degraded and local training offers negligible validation gain.

The main contributions of this paper are summarized as follows:

- We formulate vehicular FL resource scheduling as a joint utility-maximization problem that balances empirical validation loss reduction ($G_i(t)$) against V2I uplink bandwidth consumption, mixed-criticality compute contention, and L_2 parameter drift, eliminating the degenerate zero-training optimum of pure cost-minimization models.
- We design a three-state runtime policy engine (`WAIT`, `CONTINUE`, `SYNC`) coupled with a bounded non-linear exponential curve-fitting predictor (`scipy.optimize.curve_fit`) that estimates one-step-ahead marginal validation gain before committing compute cycles to additional local epochs.
- We implement the complete system in Flower and PyTorch and evaluate it across three Dirichlet non-IID regimes ($\alpha_{\text{Dir}} \in \{0.1, 0.3, 0.5\}$) against static FedAvg ($E \in \{1, 5, 10\}$) and FedProx over 250 rounds, demonstrating a 28.2% reduction in uplink traffic, a 75.3%–76.0% reduction in transmission buffer overflows (from 567 down to 136–140), and up to 73.8% lower compute contention with safety-critical tasks.

II. Related Work

Research on resource-aware vehicular federated learning spans mobility-aware transmission scheduling, joint compute–communication co-design, and asynchronous or compressed update mechanisms.

A. Mobility-Aware Transmission Scheduling

Because rapid signal fluctuations and short roadside dwell times frequently interrupt V2I model transfers, early vehicular FL work focused on link-layer reliability. Hong et al. [1] formulated upload scheduling as a Markov Decision Process (MDP) to balance transmission latency against vehicular energy budgets in CARLA co-simulations. To handle high-mobility handovers where channel state information (CSI) becomes stale, Karatas et al. [2] introduced variable-rate client selection, while Yan et al. [3] leveraged opportunistic Vehicle-to-Vehicle (V2V) relaying to salvage updates when direct V2I links drop. In all of these link-centric schedulers, however, local SGD execution is treated as a fixed-latency black box with guaranteed processor availability.

B. Joint Compute and Communication Co-Design

Subsequent frameworks incorporated onboard processing latency into the scheduling loop. Qiang et al. [4] proposed split FL to dynamically partition neural layers between vehicles and Multi-access Edge Computing (MEC) servers according to instantaneous bandwidth and compute capacity, and Ballotta et al. [5] utilized radio environment maps to schedule local iterations ahead of predicted coverage holes. Other co-design formulations pair physical resource block (PRB) allocation and client selection with Dynamic Voltage and Frequency Scaling (DVFS) or iteration bounds to trade energy against round duration [6], [7]. On the optimization side, FedProx [8] adds an L_2 proximal penalty to restrict parameter divergence when heterogeneous clients execute variable numbers of local epochs. Yet, even when scaling CPU frequency or iteration counts, these approaches assume an isolated processor allocation for FL and never model preemption or contention from co-located real-time ADAS workloads.

C. Compressed and Asynchronous Frameworks

A third line of work reduces uplink saturation by shrinking payloads or relaxing round synchronization. Liu et al. [9] and Zhang et al. [10] jointly tuned mini-batch sizes, gradient sparsification, and multi-level quantization to match instantaneous channel capacity, while Zhang et al. [12] evaluated sliding-window asynchronous parameter exchanges on embedded automotive hardware. Among asynchronous approaches, the flexible training and uploading strategy of Wu et al. [11] is closest to our work, allowing clients to defer uploads and continue local training when channel quality drops. However, Wu et al. [11] restrict vehicles to a binary choice between training and uploading; lacking both a mixed-criticality CPU model and a `WAIT` state, their scheme forces vehicles to keep running local epochs during prolonged connectivity drops even after local validation loss has completely plateaued.

D. Summary and Positioning

Table I summarizes how prior vehicular FL schedulers model computation, networking, and optimization objectives. Existing methods allocate radio blocks or static compute budgets without jointly addressing two operational questions: (i) whether an additional local epoch will produce a measurable generalization improvement on the vehicle's validation split, and (ii) whether executing that epoch will contend with co-located safety-critical workloads. Our policy engine addresses both questions by combining real-time compute-slack inspection ($1-U_i^{\text{crit}}(t)$), bounded exponential loss extrapolation, and a three-state transition policy (`WAIT`, `CONTINUE`, `SYNC`).

TABLE I
COMPARISON OF VEHICULAR FL RESOURCE ALLOCATION APPROACHES

Reference	Compute Modeling	Network Modeling	Optimization Objective
Hong et al. [1]	Implicit / fixed execution latency	Uplink transmit power and duration	Minimize weighted upload delay and energy
Karatas et al. [2]	Uniform client processing rate	Imperfect CSI, adaptive transmission rate	Accelerate convergence under link uncertainty
Yan et al. [3]	Static local compute baseline	V2V/V2I relay and opportunistic links	Maximize upload success probability
Qiang et al. [4]	CPU frequency and split layers	Dynamic channel bandwidth	Minimize total processing and transmission delay
Ballotta et al. [5]	Local iteration scheduling	Radio environment maps, path loss	Optimize training progress under round deadlines
Zhang et al. [6]	General compute resource bounds	Channel-aware client selection	Maximize final model accuracy
Pervej et al. [7]	CPU frequency and iteration bounds	PRB allocation, sojourn time	Joint energy-delay trade-off under mobility
Liu et al. [9]	Batch size scaling	Gradient compression ratio, spectrum allocation	Maximize convergence rate under delay caps
Zhang et al. [10]	Latency-coupled computation	Multi-level parameter quantization	Trade off quantization error and latency
Wu et al. [11]	Adaptive local training loops	Asynchronous upload scheduling	Minimize transmission redundancy
Zhang et al. [12]	Embedded GPU hardware evaluation	Sliding-window parameter exchange	Asynchronous convergence on embedded systems
Proposed	**Dynamic safety workload occupancy (U_i^{crit})**	**Time-varying cell capacity and throughput**	**Joint utility maximization: learning gain vs. bandwidth, model drift, and compute contention**

III. SYSTEM MODEL AND PROBLEM FORMULATION

We consider a fleet of N SDVs, indexed by $\mathcal{V} = \{1, \ldots, N\}$, coordinating with a roadside or cloud-edge FL server across discrete decision rounds $t \in \{0, 1, \ldots, T-1\}$.

A. Vehicle State Representation

At the beginning of round t, each vehicle $i \in \mathcal{V}$ observes a local operational state tuple:

$$S_i(t) = \{B_i(t),\, U_i^{\text{crit}}(t),\, D_i(t),\, G_i^{\text{prior}}(t)\} \tag{2}$$

where:

- $B_i(t) > 0$ is the instantaneous effective V2I uplink throughput available to vehicle i (in Mbps).
- $U_i^{\text{crit}}(t) \in [0, 1]$ is the fraction of onboard processor capacity occupied by higher-priority safety-critical tasks (e.g., ADAS perception and motion planning).
- $D_i(t) \geq 0$ quantifies the Euclidean (L_2) divergence between vehicle i's local model parameters and the latest broadcast global model.
- $G_i^{\text{prior}}(t) \geq 0$ records the empirical validation loss reduction achieved during the vehicle's most recent active training window.

B. Cost and Gain Components

Each scheduling round t is governed by three operational cost penalties and one learning utility reward.

1) Communication Cost: Let the binary decision variable $p_i(t) \in \{0, 1\}$ indicate whether vehicle i attempts a parameter upload at the end of round t. The fleet uplink latency cost is:

$$C_{\text{net}}(t) = \sum_{i \in \mathcal{V}} p_i(t) \frac{M_i}{B_i(t)} \tag{3}$$

where M_i denotes the serialized model payload size in megabytes (9.31 MiB for an FP32 ResNet-9 architecture).

2) Compute Contention Cost: Let $c_i(t) = e_i(t)/\rho_i \in [0, 1]$ represent the fraction of the round's compute budget allocated to FL, where $e_i(t) \in \{0, 1, \ldots, \rho_i\}$ is the number of executed local epochs and $\rho_i = 10$ is the maximum epoch capacity per round. The processor contention imposed on co-located safety workloads is modeled as the product of FL utilization and active critical occupancy:

$$C_{\text{comp}}(t) = \sum_{i \in \mathcal{V}} c_i(t) U_i^{\text{crit}}(t) = \sum_{i \in \mathcal{V}} \frac{e_i(t)}{\rho_i} U_i^{\text{crit}}(t) \tag{4}$$

3) Parameter Divergence (Model Drift): Under non-IID data distributions, extended local training causes client weights $\mathbf{w}_{\text{local},i}(t)$ to diverge from the global reference model $\mathbf{w}_{\text{global}}(t)$. We measure parameter drift directly via the L_2 norm across all L trainable tensor layers:

$$D_i(t) = \|\mathbf{w}_{\text{local},i}(t) - \mathbf{w}_{\text{global}}(t)\|_2 = \sqrt{\sum_{l=1}^{L} \|\mathbf{w}_{i,l}(t) - \mathbf{w}_{\text{global},l}(t)\|_2^2} \tag{5}$$

Summing across active vehicles yields the fleet-wide divergence cost:

$$C_{\text{drift}}(t) = \sum_{i \in \mathcal{V}} D_i(t) \tag{6}$$

4) Learning Gain and Marginal Utility: To evaluate whether local SGD updates yield useful feature learning rather than mere overfitting to already-minimized training batches, each vehicle measures cross-entropy loss on a held-out local validation split before and after local training. Let $\mathbf{w}_i^{(0)}(t)$ denote the vehicle's initial weights at round t, which equal $\mathbf{w}_{\text{global}}(t)$ following a fresh synchronization or $\mathbf{w}_{\text{local},i}(t-1)$ when resuming a multi-round `CONTINUE` trajectory. The

empirical learning gain at round t is defined as the non-negative validation loss reduction:

$$G_i(t) = \max\left(0,\ L_i(\mathbf{w}_i^{(0)}(t)) - L_i(\mathbf{w}_{\text{local},i}(t))\right) \tag{7}$$

At the client level, the net trade-off between local representation improvement and parameter divergence is captured by the base learning utility $U_i(t) = \delta G_i(t) - \gamma D_i(t)$. To evaluate this trade-off step-by-step while accounting for the incremental processor contention of each additional epoch, vehicle i tracks its validation loss trajectory after each local epoch $e \geq 2$ and estimates the one-step-ahead marginal utility of executing epoch $e{+}1$:

$$\Delta U_i(e+1) = \delta \cdot \widehat{G}_i(e+1) - \beta \frac{U_i^{\text{crit}}(t)}{\rho_i} - \gamma D_i(e) \tag{8}$$

where $\widehat{G}_i(e{+}1)$ is the one-step-ahead validation loss reduction predicted via bounded exponential curve fitting, $\beta(U_i^{\text{crit}}(t)/\rho_i)$ is the marginal compute interference of one additional epoch, $D_i(e)$ is the accumulated L_2 drift at epoch e, and $\delta, \beta, \gamma > 0$ are scalar trade-off weights.

Evaluating (8) online allows each vehicle to dynamically tailor its local training depth, avoiding both *under-training* (halting clients that are still experiencing steep validation loss drops) and *over-training* (forcing clients whose validation loss has already plateaued to waste CPU cycles and accumulate destructive parameter drift).

C. Global Fleet Objective

Over the scheduling horizon T, the fleet objective is to maximize cumulative net learning utility:

$$\max_{\{c_i(t), e_i(t), p_i(t)\}} \sum_{t=0}^{T-1} \left[\delta \sum_{i \in \mathcal{V}} G_i(t) - \alpha C_{\text{net}}(t) - \beta C_{\text{comp}}(t) - \gamma C_{\text{drift}}(t) \right] \tag{9}$$

subject to:

$$c_i(t) \leq 1 - U_i^{\text{crit}}(t), \quad \forall i, t \tag{10}$$

$$e_i(t) \leq \rho_i c_i(t), \quad \forall i, t \tag{11}$$

$$\sum_{i \in \mathcal{V}} p_i(t) M_i \leq B_{\text{cell}}(t), \quad \forall t \tag{12}$$

$$D_i(t) \leq D_i^{\max}, \quad \forall i, t \tag{13}$$

Rewarding $\delta \sum_i G_i(t)$ makes the zero-training policy $(e_i(t) = 0, p_i(t) = 0)$ suboptimal whenever positive validation gain is achievable within resource bounds, while the penalty terms prevent greedy over-training and uncoordinated channel flooding. Constraint (10) restricts FL execution to the spare processor slack left by safety-critical applications, and (11) bounds the integer epoch count accordingly. Constraint (12) limits concurrent uplink transmissions to the shared base-station buffer capacity $B_{\text{cell}}(t)$ (uploads exceeding $B_{\text{cell}}(t)$ suffer buffer overflow and are dropped), and (13) enforces a hard ceiling on local parameter divergence. Because future V2X channel states and ADAS workload spikes are unknown a priori, solving (9) offline across T rounds is intractable; we therefore decompose the decision into an online deterministic runtime policy executed independently by each vehicle.

Algorithm 1 Dynamic Runtime Policy Engine for FL Vehicle i

Require: State $S_i(t)$, local model $\mathbf{w}_i$, global reference $\mathbf{w}_{\text{global}}$, thresholds $\{B_{\text{thresh}}, G_{\text{thresh}}, \tau_{\text{continue}}, D_i^{\max}, \tau_{\text{slack}}\}$, max epochs ρ_i, max continue rounds $K_{\max}$, weights $\{\delta, \beta, \gamma\}$.

$c_i^{\max} \leftarrow \max(0, 1 - U_i^{\text{crit}}(t))$
if $c_i^{\max} < \tau_{\text{slack}}/\rho_i$ **then**
  return `WAIT` $(e_i \leftarrow 0, p_i \leftarrow 0)$ {Insufficient CPU slack}
end if
if $B_i(t) < B_{\text{thresh}}$ **and** $G_i^{\text{prior}} \leq G_{\text{thresh}}$ **then**
  return `WAIT` $(e_i \leftarrow 0, p_i \leftarrow 0)$ {Degraded link & low prior gain}
end if
$e_{\max} \leftarrow \min(\rho_i, \lfloor c_i^{\max} \cdot \rho_i \rfloor)$ $\{e_{\max} \geq \tau_{\text{slack}} = 2\}$
Initialize $e \leftarrow 0$, loss history $H \leftarrow [L_i(\mathbf{w}_i)]$
while $e < e_{\max}$ **do**
  Execute 1 local SGD epoch on $\mathbf{w}_i$ (with FedProx μ if enabled)
  $e \leftarrow e + 1$; evaluate validation loss $L_i(e)$
  Smooth: $L_{\text{smooth}} \leftarrow \theta L_i(e) + (1-\theta) H[\text{last}]$; append to H
  if $e < 2$ **or** $e \geq e_{\max}$ **then**
    continue
  end if
  Extrapolate $\widehat{L}(e+1) \leftarrow \text{BoundedExpFit}(H)$
  $\widehat{G} \leftarrow \max(0, H[\text{last}] - \widehat{L}(e+1))$
  $D_i \leftarrow \|\mathbf{w}_i - \mathbf{w}_{\text{global}}\|_2$
  $\Delta U \leftarrow \delta \widehat{G} - \beta(U_i^{\text{crit}}(t)/\rho_i) - \gamma D_i$
  if $D_i > D_i^{\max}$ **or** $\widehat{G} < G_{\text{thresh}}$ **or** $\Delta U < \tau_{\text{continue}}$ **then**
    break {Diminishing returns or drift limit}
  end if
end while
$G_i^{\text{actual}} \leftarrow \max(0, L_i(0) - L_i(e))$; $G_i^{\text{prior}} \leftarrow G_i^{\text{actual}}$
$D_i^{\text{final}} \leftarrow \|\mathbf{w}_i - \mathbf{w}_{\text{global}}\|_2$
if $B_i(t) \geq B_{\text{thresh}}$ **or** $D_i^{\text{final}} > 0.85 D_i^{\max}$ **or** (in_continue **and** $G_i^{\text{actual}} < G_{\text{thresh}}$) **or** $k_{\text{cont}} + 1 \geq K_{\max}$ **then**
  Reset $k_{\text{cont}} \leftarrow 0$, in_continue $\leftarrow$ False
  return `SYNC` $(e_i \leftarrow e, p_i \leftarrow 1)$ {Upload model update}
else
  $k_{\text{cont}} \leftarrow k_{\text{cont}} + 1$, in_continue $\leftarrow$ True
  return `CONTINUE` $(e_i \leftarrow e, p_i \leftarrow 0)$ {Persist weights locally}
end if

IV. Runtime Policy Engine

At each communication round, every scheduled vehicle inspects its local state $S_i(t)$ and executes Algorithm 1. Because true validation gain $G_i(t)$ cannot be observed prior to running local SGD, the policy operationalizes the utility rule in three sequential stages: a pre-training admission check (`WAIT`), an intra-round epoch-halting loop governed by $\Delta U_i(e{+}1)$, and a post-training transmission decision (`CONTINUE` vs. `SYNC`):

- **WAIT** ($e_i = 0, p_i = 0$)**.** The vehicle executes zero local epochs and transmits nothing. A transition to `WAIT` is triggered prior to training under two conditions: (i) safety-critical workloads leave insufficient processor slack to complete the minimum two-epoch curve-fitting window ($1 - U_i^{\text{crit}}(t) < \tau_{\text{slack}}/\rho_i$, where $\tau_{\text{slack}} = 2$), or (ii) the uplink channel is degraded ($B_i(t) < B_{\text{thresh}}$) *and* the vehicle's preceding active training window yielded negligible validation loss reduction ($G_i^{\text{prior}} \leq G_{\text{thresh}}$, evaluated after Round 1 once G_i^{prior} is initialized). Using G_i^{prior} as a pre-epoch proxy avoids burning processor cycles on an exploratory training epoch when both link quality and recent learning utility are poor, setting our state machine apart from the binary train-or-upload model of Wu et al. [11].
- **CONTINUE** ($e_i > 0, p_i = 0$)**.** When processor slack permits at least $\tau_{\text{slack}} = 2$ epochs and local training yields positive marginal utility ($\Delta U_i \geq \tau_{\text{continue}}$), but the wireless uplink is congested ($B_i(t) < B_{\text{thresh}}$), the vehicle trains locally without uploading. Rather than discarding its local progress when the server broadcasts the next round's global model, the vehicle serializes its updated weights to local persistent storage. In the subsequent round, newly broadcast global parameters are loaded into a secondary reference buffer solely to compute exact L_2 drift $D_i(t)$ and FedProx proximal gradients relative to the evolving global model while local SGD resumes from the persisted trajectory.
- **SYNC** ($e_i > 0, p_i = 1$)**.** Once local training completes and uplink throughput is sufficient ($B_i(t) \geq B_{\text{thresh}}$), the vehicle transmits its parameter update to the server. To prevent unbounded parameter divergence during prolonged connectivity drops, Algorithm 1 also forces a mandatory `SYNC` flush if accumulated drift exceeds $0.85 D_i^{\max}$, if a client in a multi-round `CONTINUE` trajectory exhausts its validation gain ($G_i^{\text{actual}} < G_{\text{thresh}}$), or if consecutive `CONTINUE` rounds reach $K_{\max} = 5$.

A. Bounded Exponential Gain Prediction

To decide whether to halt training at epoch $e \geq 2$ before reaching the hardware slack ceiling $e_{\max}$, the vehicle fits a three-parameter exponential decay model to its exponentially smoothed validation-loss trace $H = [L(0), L_{\text{smooth}}(1), \ldots, L_{\text{smooth}}(e)]$:

$$L(k) \approx L_\infty + a \cdot \exp(-b \cdot k) \tag{14}$$

Because unconstrained non-linear regression over short, noisy mini-batch validation traces ($3 \leq |H| \leq 11$) can produce ill-conditioned parameters (such as negative asymptotic loss $L_\infty < 0$ or inverted decay rates $b < 0$), we solve a box-constrained non-linear least-squares problem using Trust Region Reflective optimization (`scipy.optimize.curve_fit`):

$$\min_{L_\infty, a, b} \sum_{k=0}^{e} \left(H[k] - (L_\infty + a \cdot \exp(-b \cdot k))\right)^2 \tag{15}$$

subject to $0 \leq L_\infty \leq \min(H)$, $a \geq 0$, and $10^{-4} \leq b \leq 10.0$, initialized at $p_0 = [0.8 \min(H),\ \max(10^{-3}, H[0] - 0.8 \min(H)),\ 0.5]$. The extrapolated loss at step $e{+}1$ is clamped to $\widehat{L}(e{+}1) = \min(H[\text{last}], \max(0, L_\infty + a \exp(-b(e{+}1))))$. If the Trust Region solver fails to converge within 1 500 function evaluations, the predictor falls back to a damped linear extrapolation:

$$\widehat{L}(e+1) = \max\left(0,\ H[e] + 0.5 \cdot \min(0, H[e] - H[e-1])\right) \tag{16}$$

This bounded formulation provides stable, zero-overhead gain forecasting on resource-constrained vehicular processors without requiring auxiliary neural predictors.

V. Performance Evaluation

A. Experimental Setup and Datasets

We implement the proposed framework in PyTorch and Flower (`flwr`), executing distributed client simulations across isolated Ray worker actors. Each vehicle trains a custom ResNet-9 architecture (2 441 546 FP32 parameters, 9.31 MiB serialized payload) in which standard Batch Normalization layers are replaced with Group Normalization (8 groups per convolutional block) to prevent running-statistic divergence under non-IID data skew. Local optimization uses SGD with learning rate $\eta = 0.01$, momentum 0.9 (re-initialized per round), and mini-batch size 32.

Non-IID Vision Dataset Partitioning: We evaluate global convergence on CIFAR-10 (50 000 training images and 10 000 centralized test images) partitioned across $N = 10$ vehicles using Flower's `DirichletPartitioner` ($\alpha_{\text{Dir}} \in \{0.1, 0.3, 0.5\}$). These settings span extreme label skew ($\alpha_{\text{Dir}} = 0.1$, where individual vehicles hold 1–2 dominant object classes), moderate heterogeneity ($\alpha_{\text{Dir}} = 0.3$), and mild heterogeneity ($\alpha_{\text{Dir}} = 0.5$). Each vehicle further partitions its local dataset into 80% training samples and 20% held-out validation samples to evaluate $L_i(e)$ and $G_i(t)$.

V2X Network and Mixed-Criticality Workload Trace: At each round t, vehicles sample operational network and processor states from a 100 000-state vehicular mobility and MEC trace (`network_dataset.csv`). Instantaneous effective uplink throughput $B_i(t) \geq 0.5$ Mbps is derived from channel bandwidth (BW_{MHz}), received signal strength (RSSI_{dBm} relative to a -95 dBm noise floor), active user equipment contention (N_{UE}), and MEC server load via Shannon spectral efficiency:

$$B_i(t) = \frac{BW_{\text{MHz}} \log_2\left(1 + 10^{\max(0, \text{RSSI}_{\text{dBm}} + 95)/10}\right)}{N_{\text{UE}}} \left(1 - \frac{\text{Load}_{\%}}{100}\right) \tag{17}$$

Co-located safety-critical processor occupancy $U_i^{\text{crit}}(t) \in [0.22, 0.95]$ is computed as a function of vehicle velocity, peak traffic hours, local UE density, and active cellular handovers. The shared base-station uplink buffer envelope is modeled as $B_{\text{cell}}(t) = B_{\max}(0.8 + 0.2 \sin(t))$ with $B_{\max} = 95.0$ MiB per round.

Baselines and Hyperparameters: Over $T = 250$ communication rounds with full client participation, we compare six scheduling policies:

- Static FedAvg with fixed local epochs $E \in \{1, 5, 10\}$.
- Fixed FedProx ($E = 5$, $\mu = 0.01$) [8] to isolate the effect of proximal drift regularization under static scheduling.
- Our dynamic runtime engine in two variants: Dynamic FedAvg ($\mu = 0$) and Dynamic FedProx ($\mu = 0.01$). Utility weights were selected from a 100-trial log-uniform/uniform Pareto sweep over 50-round calibration runs: $\alpha = 0.005$, $\beta = 1.0$, $\gamma = 0.006$, $\delta = 27.0$, with operational thresholds $B_{\text{thresh}} = 20.0$ Mbps, $G_{\text{thresh}} = 0.03$, $\tau_{\text{continue}} = 0.1$, $D_i^{\max} = 20.0$, $\tau_{\text{slack}} = 2.0$, $K_{\max} = 5$, and EMA smoothing factor $\theta = 0.3$.

B. Communication Efficiency vs. Global Accuracy

To evaluate network efficiency directly against wireless resource consumption, Figures 1 and 2 plot centralized test accuracy against cumulative transmitted megabytes rather than round indices. Figure 1 tracks total bidirectional traffic (server-to-client downlink broadcasts plus client-to-server uplink uploads) on the x-axis. Note that in our accounting, every configured vehicle receives the 9.31 MB global broadcast each round (93.1 MB per round across 10 vehicles, or 23 272.2 MB over 250 rounds) so that vehicles in `CONTINUE` can update their local reference buffer $\mathbf{w}_{\text{global}}(t)$ to compute exact L_2 drift and FedProx regularization. Because V2X base stations broadcast the global model over high-capacity downlink channels while individual vehicles contend for narrow, fading V2I uplink slots, Figure 2 isolates cumulative client-to-server uplink payload on the x-axis.

As shown in Table II and Figures 1–2, static FL schedules force all 10 vehicles to attempt a 9.31 MB upload every round regardless of channel quality, generating 23 272.2 MB ($\sim$23.3 GB) of uplink traffic over 250 rounds and triggering 567 buffer overflows whenever $B_{\text{cell}}(t)$ drops below 93.1 MB. By contrast, when individual link throughput falls below $B_{\text{thresh}} = 20.0$ Mbps, our policy engine transitions vehicles into `CONTINUE` or `WAIT`, reducing cumulative uplink transfers to 16 718.7 MB for Dynamic FedAvg and 16 700.1 MB for Dynamic FedProx. This represents a **28.2% uplink bandwidth reduction (saving over 6.5 GB of wireless data)** and cuts buffer overflow drops by **75.3%–76.0%** (from 567 down to 140 and 136, respectively).

These communication savings do not compromise model convergence:

- **Extreme Non-IID ($\alpha_{\text{Dir}} = 0.1$):** Dynamic FedProx reaches **72.02%** test accuracy, matching the best static baseline (Fixed FedAvg E=5 at 72.51%) while outperforming Fixed FedProx (E=5 at 71.13%) and Fixed FedAvg E=1 (68.98%). Dynamic FedAvg achieves 70.18% under the same 28.2% uplink reduction.
- **Moderate and Mild Non-IID ($\alpha_{\text{Dir}} \in \{0.3, 0.5\}$):** Across $\alpha_{\text{Dir}} = 0.3$ and 0.5, all multi-epoch policies converge within a narrow accuracy band (76.76%–79.33% at α=0.3 and 77.88%–78.70% at α=0.5), but the dynamic policies reach the 75% accuracy threshold using approximately 35% less uplink data while eliminating three-quarters of packet drops.

Figure 3 illustrates centralized test accuracy as a function of elapsed simulation wall-clock time at $\alpha_{\text{Dir}} = 0.1$. Because Fixed E=10 unconditionally executes 10 local epochs even after local validation loss flattens, it requires over 14 500 seconds to complete 250 rounds. By truncating plateaued epochs via `BoundedExpFit` and yielding via `WAIT` during high ADAS utilization, Dynamic FedAvg and Dynamic FedProx complete all 250 rounds in $\sim$9 200 seconds, reducing total execution time by 36%.

C. Compute Contention and Model Drift

Figure 4 tracks cumulative processor contention ($\sum_t C_{\text{comp}}(t)$) imposed on co-located safety-critical workloads over 250 rounds. Because static schedulers are agnostic to $U_i^{\text{crit}}(t)$ and replay the exact same deterministic workload trace across partitions, Fixed E=1, E=5, and E=10 incur constant contention costs of 100.1, 500.3, and 1 000.6 units across all α_{Dir}, repeatedly executing heavy back-propagation loops even when ADAS tasks occupy over 80% of the processor. By contrast, our runtime engine enforces a strict slack guard ($\tau_{\text{slack}}/\rho_i = 0.20$) that immediately transitions the vehicle into `WAIT` whenever $U_i^{\text{crit}}(t) > 0.80$, while Eq. (8) halts local training early based on partition-specific loss decay. Consequently, Table II reports the range of contention across $\alpha_{\text{Dir}} \in \{0.1, 0.3, 0.5\}$ for Dynamic FedAvg (265.6–385.2) and Dynamic FedProx (261.9–383.0), representing a **61.5% to 73.8% reduction relative to Fixed** E=10 and a **23.0% to 47.7% reduction relative to Fixed** E=5.

Figure 5 plots mean per-round L_2 parameter drift ($D_i(t)$) across communication rounds. Under extreme heterogeneity ($\alpha_{\text{Dir}} = 0.1$), unfiltered 10-epoch training causes severe client divergence, with Fixed E=10 accumulating 4 474.1 in cumulative drift. Combining dynamic early stopping with FedProx ($\mu = 0.01$) restricts cumulative drift to 2 724.2 at $\alpha_{\text{Dir}} = 0.1$ and 2 149.4 at $\alpha_{\text{Dir}} = 0.5$—a **39.1% drift reduction compared to Fixed** E=10 and a **36.9% reduction compared to Fixed** E=5 **(**4 319.3**)**.

Figure 6 compares cumulative fleet utility across the three heterogeneity regimes. Inspecting the three panels reveals an important phenomenon under extreme skew ($\alpha_{\text{Dir}} = 0.1$, left panel): when static multi-epoch baselines (E=5, 10) overwrite their local models with the 10-class global aggregate at the start of every round and execute 5–10 unfiltered epochs on a narrow 1–2 class local partition, they repeatedly re-overfit to those dominant local classes. This repetitive local re-adaptation generates large per-round validation loss drops $G_i(t)$ on the skewed local split that artificially inflate raw utility sums without improving 10-class global test accuracy. By halting plateaued epochs and preserving continuous local trajectories across `CONTINUE` rounds, our dynamic policies suppress redundant local re-overfitting and converge to the same net utility band as class balance improves ($\alpha_{\text{Dir}} = 0.3$

TABLE II
COMPREHENSIVE BENCHMARK RESULTS ACROSS NON-IID DIRICHLET PARTITIONS (250 COMMUNICATION ROUNDS)

Policy / Scheme	**Centralized Test Accuracy (%)** $\alpha = 0.1$	$\alpha = 0.3$	$\alpha = 0.5$	**Cumulative Uplink Payload (MB)**	**Upload Drops**	**Compute Contention Cost** ($\sum c_i U_i^{\text{crit}}$)	**Parameter Drift Cost** ($\sum D_i$, $\alpha = 0.1/0.5$)
Fixed FedAvg ($E = 1$)	68.98	77.09	79.33	23,272.2	567	100.1	3,661.7 / 3,019.9
Fixed FedAvg ($E = 5$)	72.51	77.81	78.70	23,272.2	567	500.3	4,319.3 / 2,546.5
Fixed FedAvg ($E = 10$)	72.12	77.32	78.05	23,272.2	567	1000.6	4,474.1 / 2,494.4
Fixed FedProx ($E = 5, \mu = 0.01$)	71.13	77.10	78.63	23,272.2	567	500.3	2,956.9 / 2,016.3
Dynamic FedAvg (Ours)	70.18	76.76	78.60	16,718.7	136	265.6–385.2	4,103.2 / 2,598.7
Dynamic FedProx (Ours)	72.02	76.90	77.88	16,700.1	140	261.9–383.0	2,724.2 / 2,149.4

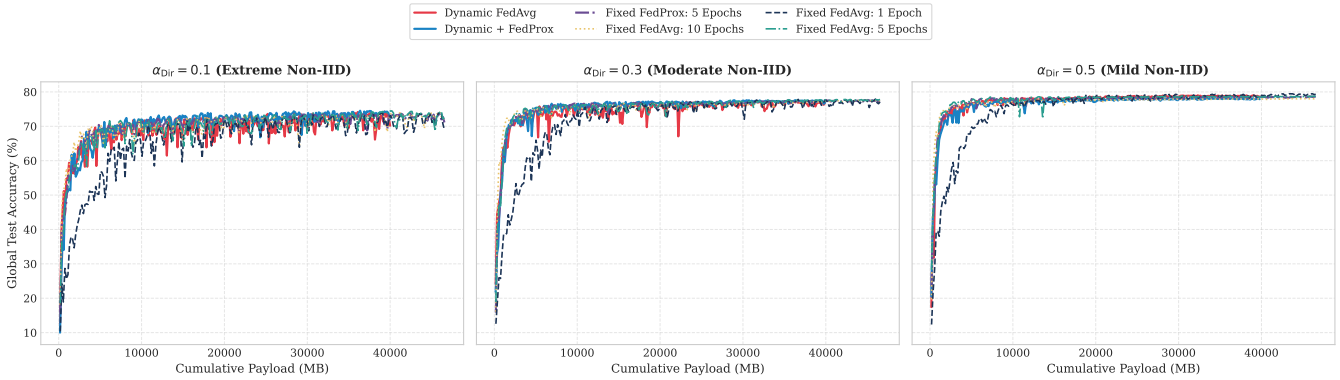


Fig. 1. Centralized test accuracy (%) on the y-axis versus cumulative bidirectional network payload (downlink broadcasts plus uplink transfers, in MB) on the x-axis across three Dirichlet non-IID regimes ($\alpha_{\text{Dir}} \in \{0.1, 0.3, 0.5\}$).

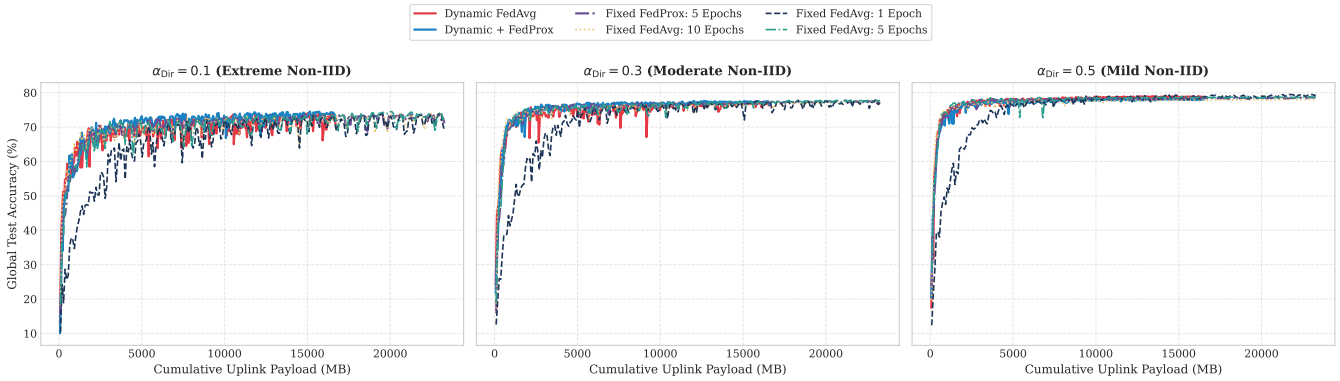


Fig. 2. Centralized test accuracy (%) on the y-axis versus cumulative *uplink-only* payload (MB) on the x-axis. Isolating client-to-server transmissions highlights the 28.2% wireless uplink reduction achieved by suppressing low-utility and congested uploads.

and 0.5), achieving equivalent global generalization at substantially lower network and compute cost.

D. Client State Dynamics

Figure 7 breaks down how the 2 500 client scheduling decisions (10 vehicles × 250 rounds) are distributed across execution states at $\alpha_{\text{Dir}} = 0.1$. Static policies operate in an unconditional transmit-or-drop mode: all 2 500 client rounds attempt an upload, resulting in 1 933 successful `SYNC` aggregations, 567 `DROPPED` buffer overflows, and zero `CONTINUE` or `WAIT` transitions.

Under our dynamic policy engine, vehicles adapt their execution state to real-time channel and processor conditions. In Dynamic FedAvg, vehicles transitioned to `CONTINUE` 474 times (468 in Dynamic FedProx) during intervals when link throughput fell below 20.0 Mbps but local validation loss was still actively decreasing. By persisting weights on disk across rounds, these clients accumulated useful representations locally instead of suffering packet drops or discarding uncommitted gradients. Another 230 decisions in Dynamic FedAvg (234 in Dynamic FedProx) transitioned to `WAIT`, capturing rounds where ADAS workloads consumed over 80% of CPU capacity or where poor link quality coincided with flat prior validation gain ($G_i^{\text{prior}} \leq 0.03$). Consequently, buffer overflows fell from 567 to 136 (Dynamic FedAvg, -76.0%) and 140 (Dynamic FedProx, -75.3%), replacing

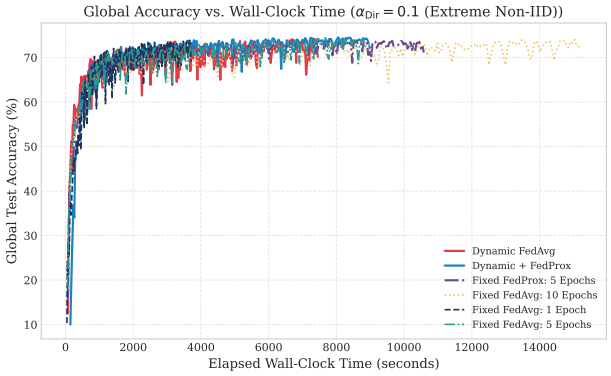


Fig. 3. Centralized test accuracy (%) on the y-axis versus elapsed simulation wall-clock time (seconds) on the x-axis ($\alpha_{\text{Dir}} = 0.1$). Dynamic epoch truncation completes 250 rounds 36% faster than high-epoch static schedules.

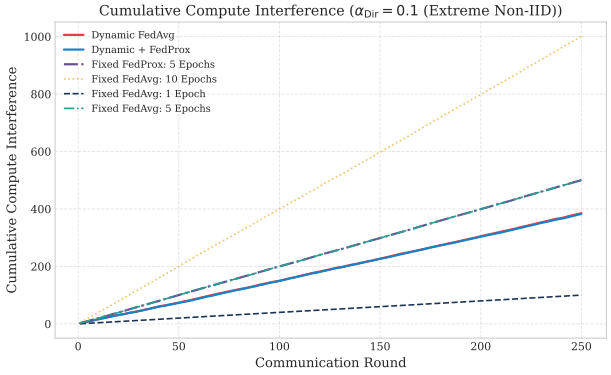


Fig. 4. Cumulative compute contention on safety-critical vehicular workloads ($\sum_t \sum_i c_i(t) U_i^{\text{crit}}(t)$) on the y-axis across 250 rounds on the x-axis ($\alpha_{\text{Dir}} = 0.1$).

wasted wireless collisions with productive local refinement or safe processor yielding.

VI. Conclusion

We presented an online runtime policy engine for federated learning in Software-Defined Vehicles that reconciles mixed-criticality onboard processor sharing with volatile V2X uplink capacity. By governing each vehicle across three execution states (`WAIT`, `CONTINUE`, and `SYNC`) using real-time compute-slack inspection and bounded exponential validation-loss extrapolation, the proposed engine prevents background FL training from starving safety-critical ADAS workloads or saturating the cellular uplink. Evaluations on Dirichlet non-IID CIFAR-10 benchmarks in Flower and PyTorch demonstrate that Dynamic FedProx matches the global test accuracy of the best static baseline under extreme heterogeneity (72.02% vs. 72.51%) while reducing uplink data transfers by 28.2%, eliminating 75.3%–76.0% of transmission buffer overflows, and cutting compute interference on safety-critical tasks by up to 73.8%. Future work will incorporate mobility trajectory prediction to anticipate cellular handovers over multi-round horizons and explore 5G NR sidelink (V2V) cluster aggregation during prolonged V2I coverage outages.

References

[1] Y. Hong, B. Lv, S. Wang, R. Wang, H. Tan, and F. C. Lau, "Fedcars: An efficient scheduling framework for in-vehicle federated learning integrated with carla-cosimulation," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 5, pp. 7997–8011, 2025.

[2] M. Karatas et al., "VR-VFL: Joint rate and client selection for vehicular federated learning under imperfect CSI," *IEEE Transactions on Mobile Computing*, early access, 2026.

[3] J. Yan, T. Chen, Y. Sun, Z. Nan, S. Zhou, and Z. Niu, "Dynamic scheduling for vehicle-to-vehicle communications enhanced federated learning," *IEEE Transactions on Wireless Communications*, pp. 9373–9390, 2024.

[4] X. Qiang, Z. Chang, Y. Hu, L. Liu, and T. Hämäläinen, "Adaptive and parallel split federated learning in vehicular edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 4591–4604, 2025.

[5] L. Ballotta, N. D. Fabbro, G. Perin, L. Schenato, M. Rossi, and G. Piro, "VREM-FL: Mobility-aware computation-scheduling co-design for vehicular federated learning," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3311–3326, 2025.

[6] X. Zhang, Z. Chang, T. Hu, W. Chen, X. Zhang, and G. Min, "Vehicle selection and resource allocation for federated learning-assisted vehicular network," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3817–3829, 2024.

[7] M. F. Pervej, R. Jin, and H. Dai, "Resource constrained vehicular edge federated learning with highly mobile connected vehicles," *IEEE Journal on Selected Areas in Communications*, pp. 1825–1844, 2023.

[8] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020, pp. 429–450.

[9] S. Liu, G. Yu, R. Yin, J. Yuan, and F. Qu, "Communication and computation efficient federated learning for internet of vehicles with a constrained latency," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 1, pp. 1038–1052, 2024.

[10] X. Zhang, W. Chen, H. Zhao, Z. Chang, and Z. Han, "Joint accuracy and latency optimization for quantized federated learning in vehicular networks," *IEEE Internet of Things Journal*, vol. 11, no. 17, pp. 28876–28890, 2024.

[11] M. Wu, M. Boban, and F. Dressler, "Flexible training and uploading strategy for asynchronous federated learning in dynamic environments," *IEEE Transactions on Mobile Computing*, pp. 12907–12921, 2024.

[12] H. Zhang, J. Bosch, and H. H. Olsson, "Real-time end-to-end federated learning: An automotive case study," in *Proc. IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2021, pp. 459–468.

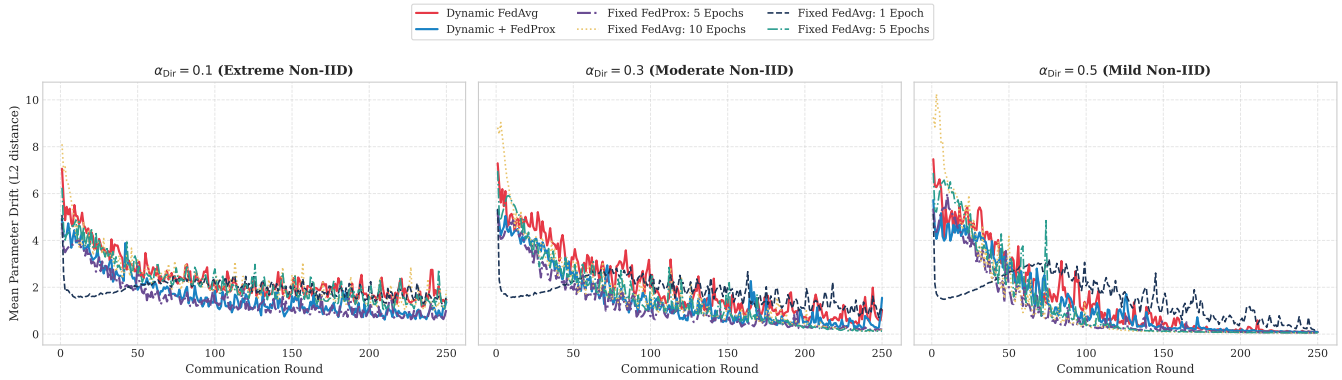


Fig. 5. Mean per-round parameter drift ($D_i(t) = \|\mathbf{w}_{\text{local},i}(t) - \mathbf{w}_{\text{global}}(t)\|_2$) on the y-axis versus communication round on the x-axis across $\alpha_{\text{Dir}} \in \{0.1, 0.3, 0.5\}$.

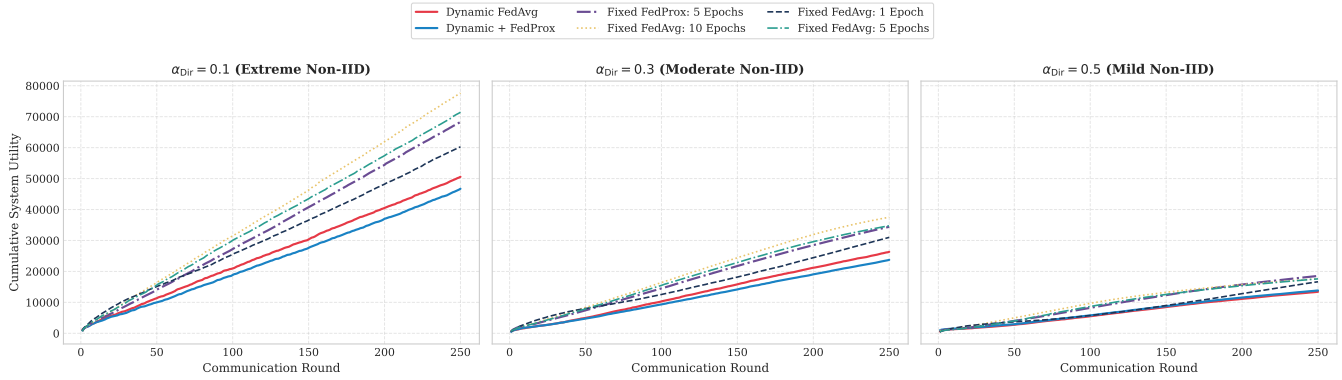


Fig. 6. Cumulative system utility ($\mathcal{U} = \sum_t [\delta \sum_i G_i(t) - \alpha C_{\text{net}}(t) - \beta C_{\text{comp}}(t) - \gamma C_{\text{drift}}(t)]$) on the y-axis versus communication round on the x-axis across Dirichlet regimes.

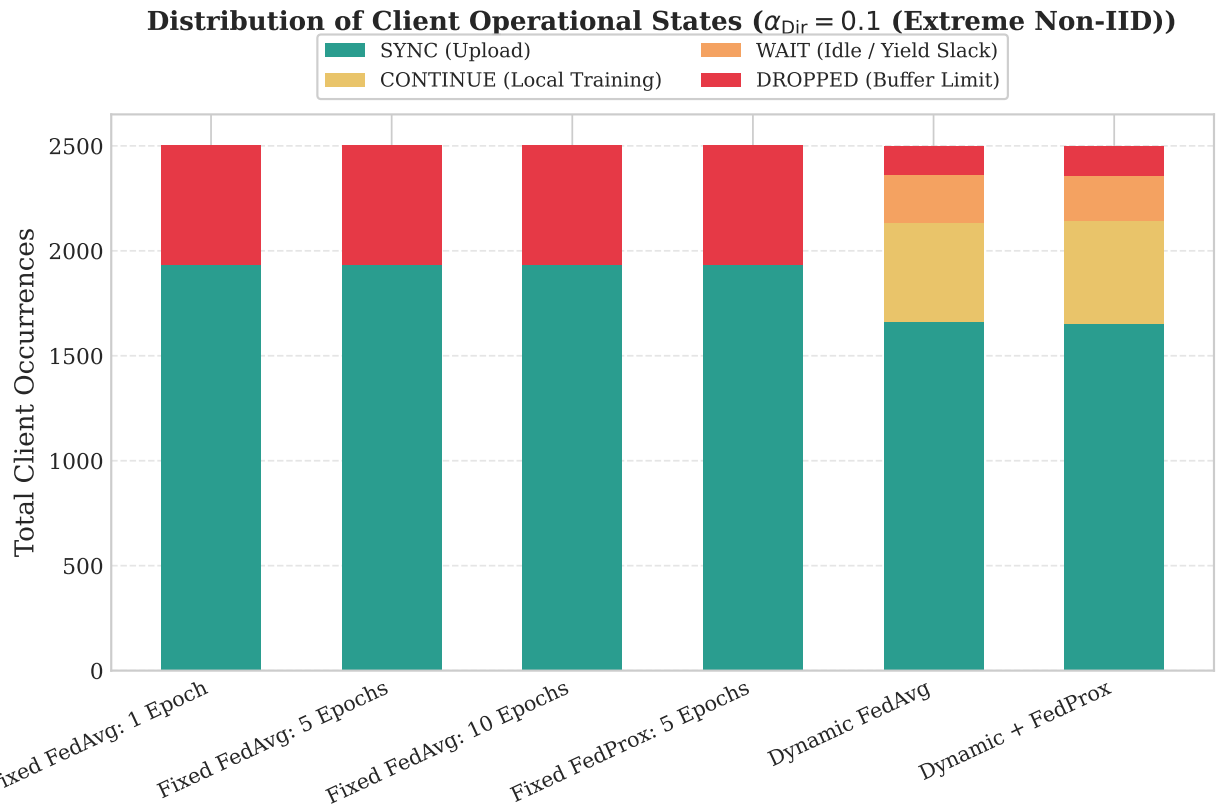


Fig. 7. Distribution of client operational states (y-axis) across 2 500 scheduling decisions over 250 rounds for each policy (x-axis) at $\alpha_{\text{Dir}} = 0.1$.